

I currently have an ESP32 that acts as a BLE beacon for a classroom proximity/geofencing system.

I want to replace ONLY the ESP32 with an Android mobile application running on the faculty's phone.

The faculty phone must perform the same BLE beacon functionality that my ESP32 currently performs.

Existing ESP32 behavior

The ESP32 currently does the following:

1. Connects to Wi-Fi.
2. Generates a random 128-bit UUID.
3. Sends the UUID, Bluetooth MAC address, device name, and device secret to Supabase.
4. After successfully sending the data to Supabase, it turns Wi-Fi completely OFF.
5. Initializes BLE.
6. Sets BLE transmit power to high power using:

```
BLEDevice::setPower(ESP_PWR_LVL_N9);
```

1. Creates a BLE advertisement.
2. Uses manufacturer-specific data.
3. The manufacturer data consists of:

```
2 bytes manufacturer identifier  
+  
16 bytes random UUID
```

Specifically:

```
uint8_t manuData[2 + 16];  
  
manuData[0] = 0xFF;  
manuData[1] = 0xFF;  
  
memcpy(manuData + 2, rotatingUUID, 16);
```

1. Advertises using:

```
ADV_TYPE_NONCONN_IND
```

1. Continuously broadcasts the BLE advertisement.

The important part is that the ESP32 is NOT connecting to other BLE devices. It is simply broadcasting an advertisement that nearby devices can detect.

Goal

Create an Android application for the faculty phone that replaces the ESP32.

The faculty phone should behave as a **BLE advertiser/beacon**.

The basic operation should be:

```
Faculty opens app
      ↓
Start Beacon
      ↓
Generate random 128-bit identifier
      ↓
Create BLE advertisement
      ↓
Broadcast continuously
```

The phone should NOT act as a BLE scanner.

It should primarily function as a BLE advertiser.

Faculty App Requirements

Create a simple Android application with:

1. Start Beacon button

When the faculty presses:

```
START BEACON
```

the application should:

1. Check that Bluetooth is enabled.
2. Check whether the phone supports BLE advertising.
3. Generate a random 128-bit UUID/identifier.

4. Create the required manufacturer-specific advertisement.
5. Start BLE advertising.
6. Display the current beacon status.

Example UI:

BLE FACULTY BEACON

Status:

● Advertising

Beacon ID:

XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX

Advertising:

Active

[STOP BEACON]

BLE Advertisement

Try to reproduce the ESP32 advertisement structure as closely as Android allows.

The ESP32 currently uses:

Manufacturer ID:

FF FF

Payload:

16-byte random UUID

Therefore investigate the Android BLE advertising API and implement the closest equivalent using:

BluetoothLeAdvertiser

AdvertiseSettings

AdvertiseData

The advertisement should be non-connectable if Android permits the equivalent configuration.

Do not establish BLE connections.

The objective is simply:

```
Faculty Phone
  ↓
BLE Advertisement
  ↓
Nearby BLE scanners can detect it
```

UUID Generation

Generate a cryptographically strong random 128-bit identifier when a new beacon session starts.

Conceptually:

```
START BEACON
  ↓
Generate 128-bit random ID
  ↓
Use it in BLE advertisement
  ↓
Keep advertising that ID
```

Do not use a permanent phone identifier as the beacon identifier.

The identifier should change when a new beacon session is started.

Supabase

The faculty application should also communicate with the existing Supabase backend.

When the faculty starts the beacon, send the relevant beacon information to Supabase.

Conceptually:

```
Faculty starts beacon
  ↓
Generate UUID
  ↓
Send UUID/session information to Supabase
```

↓
Start BLE advertising

The database should know which temporary beacon identifier belongs to the current faculty/session.

Use the existing ESP32 implementation as the reference for the Supabase request.

Important Hardware Requirement

The biggest purpose of this project is to determine whether a smartphone can replace the ESP32 as the BLE beacon.

Therefore, investigate and expose whatever Android APIs are available for controlling BLE advertising power.

The ESP32 currently uses:

ESP_PWR_LVL_N9

which is a high transmit-power setting.

Determine:

1. Whether Android allows the application to request high BLE advertising power.
2. What transmit power is actually available on different Android phones.
3. Whether the OS automatically changes/restricts advertising behavior.
4. Whether advertising continues reliably in the background.
5. Whether advertising continues when the screen is OFF.
6. Whether battery optimization affects advertising.
7. Whether different phone models have significantly different BLE range.

Do NOT assume that the phone has the same BLE transmit power or antenna performance as the ESP32.

Classroom Target

The intended use is:

One faculty phone
↓
BLE beacon



Classroom containing approximately 120 students

The faculty phone should ideally provide enough BLE advertising range to cover an entire classroom.

The application should therefore prioritize:

- Reliable advertising
- Maximum permitted BLE transmit power
- Appropriate advertising interval
- Stable background operation
- Low probability of advertisement interruption

Do not design the system around Bluetooth connections.

It is a broadcast beacon.

Testing Mode

Add a testing/debug screen that displays:

```
BLE supported: YES/NO
BLE advertising supported: YES/NO
Bluetooth enabled: YES/NO
Advertising status: ACTIVE/INACTIVE
Beacon UUID: XXXXXXXX...
Advertising mode: ...
Requested TX power: ...
```

If Android provides information about the actual advertising capabilities/power, display it.

Also provide logs showing:

```
Beacon started
UUID generated
Advertisement configured
Advertisement started
Advertisement stopped
Advertising error
```

Final Objective

The final faculty application should essentially replace this ESP32 architecture:

```
ESP32
|
|— Generate random UUID
|
|— Send UUID to Supabase
|
|— BLE high-power configuration
|
|— Manufacturer-specific advertisement
|
|— Continuous BLE broadcast
```

with:

```
Faculty Android Phone
|
|— Generate random UUID
|
|— Send UUID to Supabase
|
|— Request highest practical BLE advertising power
|
|— Manufacturer-specific advertisement
|
|— Continuous BLE broadcast
```

Do not implement the student scanner, attendance logic, RSSI processing, or student application.

The only objective at this stage is:

Make the faculty Android phone function as reliably and powerfully as possible as a replacement for the existing ESP32 BLE beacon, while preserving the existing advertisement/UUID concept.